connections and more. You should *release* resources as soon as they're no longer needed. This practice ensures that other applications can use the resources. Python's **`with` statement**:

- acquires a resource (in this case, the file object for `accounts.txt`) and assigns its corresponding object to a variable (`accounts` in this example),
- allows the application to use the resource via that variable, and
- calls the resource object's `close` method to release the resource when program control reaches the end of the `with` statement's suite.

Built-In Function `open`

The built-in **`open` function** opens the file `accounts.txt` and associates it with a file object. The `mode` argument specifies the **file-open mode**, indicating whether to open a file for reading from the file, for writing to the file or both. The mode `'w'` opens the file for *writing*, creating the file if it does not exist. If you do not specify a path to the file, Python creates it in the current folder (`ch09`). Be careful—opening a file for writing *deletes* all the existing data in the file. By convention, the **`.txt` file extension** indicates a plain text file.

Writing to the File

The `with` statement assigns the object returned by `open` to the variable `accounts` in the **`as` clause**. In the `with` statement's suite, we use the variable `accounts` to interact with the file. In this case, we call the file object's **`write` method** five times to write five records to the file, each as a separate line of text ending in a newline. At the end of the `with` statement's suite, the `with` statement *implicitly* calls the file object's **`close`** method to close the file.

Contents of `accounts.txt` File

After executing the previous snippet, your `ch09` directory contains the file `accounts.txt` with the following contents, which you can view by opening the file in a text editor:

```
100 Jones 24.98
200 Doe 345.67
300 White 0.00
400 Stone -42.16
500 Rich 224.62
```

In the next section, you'll read the file and display its contents.

9.3.2 Reading Data from a Text File

We just created the text file `accounts.txt` and wrote data to it. Now let's read that data from the file sequentially from beginning to end. The following session reads records from the file `accounts.txt` and displays the contents of each record in columns with the `Account` and `Name` columns *left aligned* and the `Balance` column *right aligned*, so the decimal points align vertically:

lick here to view code image

```
In [1]: with open('accounts.txt', mode='r')   as accounts:
   ...:     print(f'{"Account":<10}{"Name":<10}{"Balance":>10}')
   ...:     for record in accounts:
   ...:         account, name, balance = record.split()
   ...:         print(f'{account:<10}{name:<10}{balance:>10}')
   ...:
Account   Name         Balance
100       Jones          24.98
200       Doe           345.67
300       White           0.00
400       Stone         -42.16
500       Rich          224.62
```

If the contents of a file should not be modified, open the file for reading only. This prevents the program from accidentally modifying the file. You open a file for reading by passing the `'r'` file-open mode as function `open`'s second argument. If you do not specify the folder in which to store the file, `open` assumes the file is in the current folder.

Iterating through a file object, as shown in the preceding `for` statement, reads one line at a time from the file and returns it as a string. For each `record` (that is, line) in the file, string method `split` returns tokens in the line as a list, which we unpack into the variables `account`, `name` and `balance`. [1] The last statement in the `for` statement's suite displays these variables in columns using field widths.

[1] When splitting strings on spaces (the default), `split` automatically discards the newline character.

File Method `readlines`

C

The file object's **`readlines`** method also can be used to read an *entire* text file. The method returns each line as a string in a list of strings. For small files, this works well, but iterating over the lines in a file object, as shown above, can be more efficient. [2] Calling `readlines` for a large file can be a time-consuming operation, which must complete before you can begin using the list of strings. Using the file object in a `for` statement enables your program to process each text line as it's read.

[2] ttps://docs.python.org/3/tutorial/inputoutput.html#methods-f-file-objects.

Seeking to a Specific File Position

While reading through a file, the system maintains a **file-position pointer** representing the location of the next character to read. Sometimes it's necessary to process a file sequentially from the beginning *several times* during a program's execution. Each time, you must reposition the file-position pointer to the beginning of the file, which you can do either by closing and reopening the file, or by calling the file object's **`seek`** method, as in

```
file_object.seek(0)
```

The latter approach is faster.

9.4 UPDATING TEXT FILES

Formatted data written to a text file cannot be modified without the risk of destroying other data. If the name `'White'` needs to be changed to `'Williams'` in `accounts.txt`, the old name cannot simply be overwritten. The original record for `White` is stored as

```
300 White 0.00
```

If you overwrite the name `'White'` with the name `'Williams'`, the record becomes

```
300 Williams00
```

The new last name contains three more characters than the original one, so the characters beyond the second "`i`" in `'Williams'` overwrite other characters in the line. The problem is that in the formatted input–output model, records and their fields

can vary in size. For example, 7, 14, –117, 2074 and 27383 are all integers and are stored in the same number of "raw data" bytes internally (typically 4 or 8 bytes in today's systems). However, when these integers are output as formatted text, they become different-sized fields. For example, 7 is one character, 14 is two characters and 27383 is five characters.

To make the preceding name change, we can:

- copy the records before `300 White 0.00` into a temporary file,
- write the updated and correctly formatted record for account 300 to this file,
- copy the records after `300 White 0.00` to the temporary file,
- delete the old file and
- rename the temporary file to use the original file's name.

This can be cumbersome because it requires processing *every* record in the file, even if you need to update only one record. Updating a file as described above is more efficient when an application needs to update many records in one pass of the file. [3]

[3] n the chapter, Big Data: Hadoop, Spark, NoSQL and IoT, youll see that database systems solve this update in place problem efficiently.

Updating `accounts.txt`

Let's use a `with` statement to update the `accounts.txt` file to change account 300's name from `'White'` to `'Williams'` as described above:

lick here to view code image

```
 n [1]: accounts = open('accounts.txt',   'r')

In [2]: temp_file = open('temp_file.txt',   'w')

In [3]: with accounts,   temp_file:
   ...:     for record in accounts:
   ...:         account, name, balance = record.split()
   ...:         if account != '300':
   ...:             temp_file.write(record)
   ...:         else:
   ...:             new_record = ' '.join([account,   'Williams', balance]
   ...:             temp_file.write(new_record + '\n')
```

C

```
    ...:
```

or readability, we opened the file objects (snippets `[1]` and `[2]`), then specified their variable names in the first line of snippet `[3]`. This `with` statement manages two resource objects, specified in a comma-separated list after `with`. The `for` statement unpacks each record into `account`, `name` and `balance`. If the `account` is not `'300'`, we write `record` (which contains a newline) to `temp_file`. Otherwise, we assemble the new record containing `'Williams'` in place of `'White'` and write it to the file. After snippet `[3]`, `temp_file.txt` contains:

```
100 Jones 24.98
200 Doe 345.67
300 Williams 0.00
400 Stone -42.16
500 Rich 224.62
```

`os` Module File-Processing Functions

At this point, we have the old `accounts.txt` file and the new `temp_file.txt`. To complete the update, let's delete the old `accounts.txt` file, then rename `temp_file.txt` as `accounts.txt`. The **`os` module** [4] provides functions for interacting with the operating system, including several that manipulate your system's files and directories. Now that we've created the temporary file, let's use the **`remove` function** [5] to delete the original file:

[4] `ttps://docs.python.org/3/library/os.html`.

[5] Use `remove` with cautionit does not warn you that youre *permanently* deleting the file.

lick here to view code image

```
In [4]: import os

In [5]: os.remove('accounts.txt')
```

Next, let's use the **`rename` function** to rename the temporary file as `'accounts.txt'`:

lick here to view code image

E

```
In [6]: os.rename('temp_file.txt',   'accounts.txt')
```

9.5 SERIALIZATION WITH JSON

Many libraries we'll use to interact with cloud-based services, such as Twitter, IBM Watson and others, communicate with your applications via JSON objects. **JSON (JavaScript Object Notation)** is a text-based, human-and-computer-readable, data-interchange format used to represent objects as collections of name–value pairs. JSON can even represent objects of custom classes like those you'll build in the next chapter.

JSON has become the preferred data format for transmitting objects across platforms. This is especially true for invoking cloud-based web services, which are functions and methods that you call over the Internet. You'll become proficient at working with JSON data. In the "Data Mining Twitter" chapter, you'll access JSON objects containing tweets and their metadata. In the "IBM Watson and Cognitive Computing" chapter, you'll access data in the JSON responses returned by Watson services. In the " ig Data: adoop, Spark, NoSQL and IoT" chapter, we'll store JSON tweet objects that we obtain from Twitter in MongoDB, a popular NoSQL database. In that chapter, we'll also work with other web services that send and receive data as JSON objects.

JSON Data Format

JSON objects are similar to Python dictionaries. Each JSON object contains a comma-separated list of *property names* and *values*, in curly braces. For example, the following key–value pairs might represent a client record:

```
{"account": 100, "name": "Jones", "balance": 24.98}
```

JSON also supports arrays which, like Python lists, are comma-separated values in square brackets. For example, the following is an acceptable JSON array of numbers:

```
[100, 200, 300]
```

Values in JSON objects and arrays can be:

- strings in *double quotes* (like `"Jones"`),
- numbers (like `100` or `24.98`),

- JSON Boolean values (represented as `true` or `false` in JSON),
- `null` (to represent no value, like `None` in Python),
- arrays (like `[100, 200, 300]`), and
- other JSON objects.

Python Standard Library Module `json`

The **`json` module** enables you to convert objects to JSON (JavaScript Object Notation) text format. This is known as **serializing** the data. Consider the following dictionary, which contains one key–value pair consisting of the key `'accounts'` with its associated value being a list of dictionaries representing two accounts. Each account dictionary contains three key–value pairs for the account number, name and balance:

lick here to view code image

```
In [1]: accounts_dict = {'accounts':   [
   ...:     {'account': 100, 'name': 'Jones', 'balance': 24.98},
   ...:     {'account': 200, 'name': 'Doe', 'balance': 345.67}]}
```

Serializing an Object to JSON

Let's write that object in JSON format to a file:

lick here to view code image

```
In [2]: import json

In [3]: with open('accounts.json', 'w') as accounts:
   ...:     json.dump(accounts_dict, accounts)
   ...:
```

Snippet `[3]` opens the file `accounts.json` and uses the `json` module's **`dump` function** to serialize the dictionary `accounts_dict` into the file. The resulting file contains the following text, which we reformatted slightly for readability:

lick here to view code image

```
{"accounts":
   [{"account": 100, "name": "Jones", "balance": 24.98},
    {"account": 200, "name": "Doe", "balance": 345.67}]}
```

C

Note that JSON delimits strings with *double-quote characters*.

Deserializing the JSON Text

The `json` module's **`load` function** reads the entire JSON contents of its file object argument and converts the JSON into a Python object. This is known as **deserializing** the data. Let's reconstruct the original Python object from this JSON text:

lick here to view code image

```
In [4]: with open('accounts.json', 'r') as accounts:
   ...:     accounts_json = json.load(accounts)
   ...:
   ...:
```

We can now interact with the loaded object. For example, we can display the dictionary:

lick here to view code image

```
In [5]: accounts_json
Out[5]:
{'accounts': [{'account': 100, 'name': 'Jones', 'balance':   24.98},
  {'account': 200, 'name': 'Doe', 'balance': 345.67}]}
```

As you'd expect, you can access the dictionary's contents. Let's get the list of dictionaries associated with the `'accounts'` key:

lick here to view code image

```
In [6]: accounts_json['accounts']
Out[6]:
[{'account': 100, 'name': 'Jones', 'balance': 24.98},
 {'account': 200, 'name': 'Doe', 'balance': 345.67}]
```

Now, let's get the individual account dictionaries:

lick here to view code image

```
In [7]: accounts_json['accounts'][0]
Out[7]: {'account': 100, 'name': 'Jones', 'balance': 24.98}

In [8]: accounts_json['accounts'][1]
```

C

```
Out[8]: {'account': 200, 'name': 'Doe', 'balance': 345.67}
```

Though we did not do so here, you can modify the dictionary as well. For example, you could add accounts to or remove accounts from the list, then write the dictionary back into the JSON file.

Displaying the JSON Text

The `json` module's **`dumps` function** (`dumps` is short for "dump string") returns a Python string representation of an object in JSON format. Using `dumps` with `load`, you can read the JSON from the file and display it in a nicely indented format—sometimes called "pretty printing" the JSON. When the `dumps` function call includes the `indent` keyword argument, the string contains newline characters and indentation for pretty printing—you also can use `indent` with the `dump` function when writing to a file:

lick here to view code image

```
In [9]: with open('accounts.json', 'r') as accounts:
   ...:     print(json.dumps(json.load(accounts), indent=4))
   ...:
{
    "accounts": [
        {
            "account": 100,
            "name": "Jones",
            "balance": 24.98
        },
        {
            "account": 200,
            "name": "Doe",
            "balance": 345.67
        }
    ]
}
```

9.6 FOCUS ON SECURITY: `PICKLE` SERIALIZATION AND DESERIALIZATION

The Python Standard Library's **`pickle` module** can serialize objects into in a Python-specific data format. **Caution: The Python documentation provides the following warnings about `pickle`:**

- "Pickle files can be hacked. If you receive a raw pickle file over the network, don't trust it! It could have malicious code in it, that would run arbitrary Python when

C

ou try to de-pickle it. However, if you are doing your own pickle writing and reading, you're safe (provided no one else has access to the pickle file, of course.)" [6]

[6] ttps://wiki.python.org/moin/UsingPickle.

- "Pickle is a protocol which allows the serialization of arbitrarily complex Python objects. As such, it is specific to Python and cannot be used to communicate with applications written in other languages. It is also insecure by default: deserializing pickle data coming from an untrusted source can execute arbitrary code, if the data was crafted by a skilled attacker." [7]

[7] ttps://docs.python.org/3/tutorial/inputoutput.html#reading-nd-writing-files.

We do not recommend using `pickle`, but it's been used for many years, so you're likely to encounter it in **legacy code**—old code that's often no longer supported.

9.7 ADDITIONAL NOTES REGARDING FILES

The following table summarizes the various file-open modes for text files, including the modes for reading and writing we've introduced. The *writing* and *appending* modes create the file if it does not exist. The *reading* modes raise a `FileNotFoundError` if the file does not exist. Each text-file mode has a corresponding binary-file mode specified with `b`, as in `'rb'` or `'wb+'`. You'd use these modes, for example, if you were reading or writing binary files, such as images, audio, video, compressed ZIP files and many other popular custom file formats.

Mode	Description
`'r'`	Open a text file for reading. This is the default if you do not specify the file-open mode when you call open.
`'w'`	Open a text file for writing. Existing file contents are *deleted*.

`'a'`	Open a text file for appending at the end, creating the file if it does not exist. New data is written at the end of the file.
`'r+'`	Open a text file reading and writing.
`'w+'`	Open a text file reading and writing. Existing file contents are *deleted*.
`'a+'`	Open a text file reading and appending at the end. New data is written at the end of the file. If the file does not exist, it is created.

Other File Object Methods

Here are a few more useful file-object methods.

- For a text file, the `read` method returns a string containing the number of characters specified by the method's integer argument. For a binary file, the method returns the specified number of bytes. If no argument is specified, the method returns the entire contents of the file.
- The `readline` method returns one line of text as a string, including the newline character if there is one. This method returns an empty string when it encounters the end of the file.
- The `writelines` method receives a list of strings and writes its contents to a file.

The classes that Python uses to create file objects are defined in the Python Standard Library's **`io` module** (`ttps://docs.python.org/3/library/io.html`).

9.8 HANDLING EXCEPTIONS

Various types of exceptions can occur when you work with files, including:

- A `FileNotFoundError` occurs if you attempt to open a non-existent file for reading with the `'r'` or `'r+'` modes.

h

- A **`PermissionsError`** occurs if you attempt an operation for which you do not have permission. This might occur if you try to open a file that your account is not allowed to access or create a file in a folder where your account does not have permission to write, such as where your computer's operating system is stored.
- A `ValueError` (with the error message `'I/O operation on closed file.'`) occurs when you attempt to write to a file that has already been closed.

9.8.1 Division by Zero and Invalid Input

Let's revisit two exceptions that you saw earlier in the book.

Division By Zero

Recall that attempting to divide by `0` results in a `ZeroDivisionError`:

lick here to view code image

```
In [1]: 10 / 0
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last
 ipython-input-1-a243dfbf119d> in <module>()
----> 1 10 / 0

ZeroDivisionError: division   by zero

In [2]:
```

In this case, the interpreter is said to **raise an exception** of type `ZeroDivisionError`. When an exception is raised in IPython, it:

- terminates the snippet,
- displays the exception's traceback, then
- shows the next `In []` prompt so you can input the next snippet.

If an exception occurs in a script, it terminates and IPython displays the traceback.

Invalid Input

Recall that the `int` function raises a `Value-Error` if you attempt to convert to an

nteger a string (like `'hello'`) that does not represent a number:

lick here to view code image

```
In [2]: value = int(input('Enter an   integer: '))
Enter an integer: hello
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last
 ipython-input-2-b521605464d6> in <module>()
----> 1 value = int(input('Enter   an integer: '))

ValueError: invalid literal   for int() with base 10: 'hello'

In [3]:
```

9.8.2 `try` Statements

Now let's see how to *handle* these exceptions so that you can enable code to continue processing. Consider the following script and sample execution. Its loop attempts to read two integers from the user, then display the first number divided by the second. The script uses exception handling to catch and handle (i.e., deal with) any `ZeroDivisionError`s and `ValueError`s that arise—in this case, allowing the user to re-enter the input.

lick here to view code image

```
1 # dividebyzero.py
2 """Simple exception handling   example."""
3
4 while True:
5     # attempt to convert and divide values
6     try:
7         number1   = int(input('Enter numerator: '))
8         number2   = int(input('Enter denominator: '))
9         result   = number1 / number2
10     except ValueError:  # tried to   convert non-numeric value to int
11         print('You must enter two integers\n')
12     except ZeroDivisionError:  #   denominator was 0
13         print('Attempted to divide by zero\n')
14     else:  # executes only if no   exceptions occur
15         print(f'{number1:.3f} /   {number2:.3f} = {result:.3f}')
16         break  # terminate the loop
```

lick here to view code image

```
Enter numerator: 100
Enter denominator: 0
Attempted to divide by zero

Enter numerator: 100
Enter denominator: hello
You must enter two integers

Enter numerator: 100
Enter denominator: 7
100.000 / 7.000 = 14.286
```

`try` Clause

Python uses **`try` statements** (like lines 6–16) to enable exception handling. The `try` statement's **`try` clause** (lines 6–9) begins with keyword `try`, followed by a colon (`:`) and a suite of statements that *might* raise exceptions.

`except` Clause

A `try` clause may be followed by one or more **`except` clauses** (lines 10–11 and 12–13) that immediately follow the `try` clause's suite. These also are known as *exception handlers*. Each `except` clause specifies the type of exception it handles. In this example, each exception handler just displays a message indicating the problem that occurred.

`else` Clause

After the last `except` clause, an optional **`else` clause** (lines 14–16) specifies code that should execute only if the code in the `try` suite did not raise exceptions. If no exceptions occur in this example's `try` suite, line 15 displays the division result and line 16 terminates the loop.

Flow of Control for a `ZeroDivisionError`

Now let's consider this example's flow of control, based on the first three lines of the sample output:

- First, the user enters `100` for the numerator in response to line 7 in the `try` suite.
- Next, the user enters `0` for the denominator in response to line 8 in the `try` suite.
- At this point, we have two integer values, so line 9 attempts to divide `100` by `0`,

ausing Python to raise a `ZeroDivisionError`. The point in the program at which an exception occurs is often referred to as the **raise point**.

When an exception occurs in a `try` suite, it terminates immediately. If there are any `except` handlers following the `try` suite, program control transfers to the first one. If there are no `except` handlers, a process called *stack unwinding* occurs, which we discuss later in the chapter.

In this example, there *are* `except` handlers, so the interpreter searches for the *first one* that matches the type of the raised exception:

- The `except` clause at lines 10–11 handles `ValueError`s. This does not match the type `ZeroDivisionError`, so that `except` clause's suite does not execute and program control transfers to the next `except` handler.
- The `except` clause at lines 12–13 handles `ZeroDivisionError`s. This *is* a match, so that `except` clause's suite executes, displaying `"Attempted to divide by zero"`.

When an `except` clause successfully handles the exception, program execution resumes with the `finally` clause (if there is one), then with the next statement after the `try` statement. In this example, we reach the end of the loop, so execution resumes with the next loop iteration. Note that after an exception is handled, program control does *not* return to the raise point. Rather, control resumes after the `try` statement. We'll discuss the `finally` clause shortly.

Flow of Control for a `ValueError`

Now let's consider the flow of control, based on the next three lines of the sample output:

- First, the user enters `100` for the numerator in response to line 7 in the `try` suite.
- Next, the user enters `hello` for the denominator in response to line 8 in the `try` suite. The input is not a valid integer, so the `int` function raises a `ValueError`.

The exception terminates the `try` suite and program control transfers to the first `except` handler. In this case, the `except` clause at lines 10–11 is a match, so its suite executes, displaying `"You must enter two integers"`. Then, program execution

c

resumes with the next statement after the `try` statement. Again, that's the end of the loop, so execution resumes with the next loop iteration.

Flow of Control for a Successful Division

Now let's consider the flow of control, based on the last three lines of the sample output:

- First, the user enters `100` for the numerator in response to line 7 in the `try` suite.
- Next, the user enters `7` for the denominator in response to line 8 in the `try` suite.
- At this point, we have two valid integer values and the denominator is not `0`, so line 9 successfully divides `100` by `7`.

When no exceptions occur in the `try` suite, program execution resumes with the `else` clause (if there is one); otherwise, program execution resumes with the next statement after the `try` statement. In this example's `else` clause, we display the division result, then terminate the loop, and the program terminates.

9.8.3 Catching Multiple Exceptions in One `except` Clause

It's relatively common for a `try` clause to be followed by several `except` clauses to handle various types of exceptions. If several `except` suites are identical, you can catch those exception types by specifying them as a tuple in a *single* `except` handler, as in:

```
except (type1, type2, ...) as variable_name:
```

The `as` clause is optional. Typically, programs do not need to reference the caught exception object directly. If you do, you can use the variable in the `as` clause to reference the exception object in the `except` suite.

9.8.4 What Exceptions Does a Function or Method Raise?

Exceptions may surface via statements in a `try` suite, via functions or methods called directly or indirectly from a `try` suite, or via the Python interpreter as it executes the code (for example, `ZeroDivisionError`s).

Before using any function or method, read its online API documentation, which specifies what exceptions are thrown (if any) by the function or method and indicates

easons why such exceptions may occur. Next, read the online API documentation for each exception type to see potential reasons why such an exception occurs.

9.8.5 What Code Should Be Placed in a `try` Suite?

Place in a `try` suite a significant logical section of a program in which several statements can raise exceptions, rather than wrapping a separate `try` statement around every statement that raises an exception. However, for proper exception-handling granularity, each `try` statement should enclose a section of code small enough that, when an exception occurs, the specific context is known and the `except` handlers can process the exception properly. If many statements in a `try` suite raise the same exception types, multiple `try` statements may be required to determine each exception's context.

9.9 `FINALLY` CLAUSE

Operating systems typically can prevent more than one program from manipulating a file at once. When a program finishes processing a file, the program should close it to release the resource so other programs can access it. Closing the file helps prevent a **resource leak**.

The `finally` Clause of the `try` Statement

A `try` statement may have a `finally` clause after any `except` clauses or the `else` clause. The **`finally`** clause is guaranteed to execute. [8] In other languages that have `finally`, this makes the `finally` suite an ideal location to place resource-deallocation code for resources acquired in the corresponding `try` suite. In Python, we prefer the `with` statement for this purpose and place other kinds of "clean up" code in the `finally` suite.

[8] The only reason a `finally` suite will not execute if program control enters the corresponding `try` suite is if the application terminates first, for example by calling the `sys` modules `exit` function.

Example

The following IPython session demonstrates that the `finally` clause always executes, regardless of whether an exception occurs in the corresponding `try` suite. First, let's consider a `try` statement in which no exceptions occur in the `try` suite:

lick here to view code image

€

```
In [1]: try:
   ...:     print('try suite with no   exceptions raised')
   ...: except:
   ...:     print('this will not   execute')
   ...: else:
   ...:     print('else executes   because no exceptions in the try suite'
   ...: finally:
   ...:     print('finally always   executes')
   ...:
try suite with no exceptions raised
else executes because no exceptions in the try suite
finally always executes

In [2]:
```

The preceding `try` suite displays a message but does not raise any exceptions. When program control successfully reaches the end of the `try` suite, the `except` clause is skipped, the `else` clause executes and the `finally` clause displays a message showing that it always executes. When the `finally` clause terminates, program control continues with the next statement after the `try` statement. In an IPython session, the next `In []` prompt appears.

Now let's consider a `try` statement in which an exception occurs in the `try` suite:

lick here to view code image

```
In [2]: try:
   ...:     print('try suite that   raises an exception')
   ...:     int('hello')
   ...:     print('this will not   execute')
   ...: except ValueError:
   ...:     print('a ValueError   occurred')
   ...: else:
   ...:     print('else will not   execute because an exception occurred')
   ...: finally:
   ...:     print('finally always   executes')
   ...:
try suite that raises an exception
a ValueError occurred
finally always executes

In [3]:
```

This `try` suite begins by displaying a message. The second statement attempts to convert the string `'hello'` to an integer, which causes the `int` function to raise a

C

`ValueError`. The `try` suite immediately terminates, skipping its last `print` statement. The `except` clause catches the `ValueError` exception and displays a message. The `else` clause does not execute because an exception occurred. Then, the `finally` clause displays a message showing that it always executes. When the `finally` clause terminates, program control continues with the next statement after the try statement. In an IPython session, the next `In []` prompt appears.

Combining `with` Statements and `try` `except` Statements

Most resources that require explicit release, such as files, network connections and database connections, have potential exceptions associated with processing those resources. For example, a program that processes a file might raise `IOError`s. For this reason, *robust* file-processing code normally appears in a `try` suite containing a `with` statement to guarantee that the resource gets released. The code is in a `try` suite, so you can catch in `except` handlers any exceptions that occur and you do not need a `finally` clause because the `with` statement handles resource deallocation.

To demonstrate this, first let's assume you're asking the user to supply the name of a file and they provide that name incorrectly, such as `gradez.txt` rather than the file we created earlier `grades.txt`. In this case, the `open` call raises a `FileNotFoundError` by attempting to open a non-existent file:

lick here to view code image

```
In [3]: open('gradez.txt')
---------------------------------------------------------------------------
FileNotFoundError                         Traceback (most recent call last
 ipython-input-3-b7f41b2d5969> in <module>()
----> 1 open('gradez.txt')

FileNotFoundError: [Errno 2]   No such file or directory: 'gradez.txt'
```

To catch exceptions like `FileNotFoundError` that occur when you try to open a file for reading, wrap the `with` statement in a `try` suite, as in:

lick here to view code image

```
In [4]: try:
   ...:     with open('gradez.txt', 'r') as accounts:
   ...:         print(f'{"ID":<3}{"Name":<7}{"Grade"}')
   ...:         for record in accounts:
```

e

```
    ...:             student_id, name, grade =   record.split()
    ...:             print(f'{student_id:<3}{name:<7}{grade}')
    ...: except   FileNotFoundError:
    ...:     print('The file name you   specified does not exist')
    ...:
The file name you specified does not exist
```

9.10 EXPLICITLY RAISING AN EXCEPTION

You've seen various exceptions raised by your Python code. Sometimes you might need to write functions that raise exceptions to inform callers of errors that occur. The **`raise`** statement explicitly raises an exception. The simplest form of the `raise` statement is

raise *ExceptionClassName*

The `raise` statement creates an object of the specified exception class. Optionally, the exception class name may be followed by parentheses containing arguments to initialize the exception object—typically to provide a custom error message string. Code that raises an exception first should release any resources acquired before the exception occurred. In the next section, we'll show an example of raising an exception.

In most cases, when you need to raise an exception, it's recommended that you use one of Python's many built-in exception types [9] listed at:

[9] You may be tempted to create custom exception classes that are specific to your application. Well say more about custom exceptions in the next chapter.

ttps://docs.python.org/3/library/exceptions.html

9.11 (OPTIONAL) STACK UNWINDING AND TRACEBACKS

Each exception object stores information indicating the precise series of function calls that led to the exception. This is helpful when debugging your code. Consider the following function definitions—`function1` calls `function2` and `function2` raises an `Exception`:

lick here to view code image

```
In [1]: def function1():
   ...:     function2()
```

```
    ...:

In [2]: def function2():
    ...:     raise Exception('An exception occurred')
    ...:
```

Calling `function1` results in the following traceback. For emphasis, we placed in bold the parts of the traceback indicating the lines of code that led to the exception:

lick here to view code image

```
In [3]: function1()
---------------------------------------------------------------------------
Exception                                 Traceback (most recent call last
 ipython-input-3-c0b3cafe2087> in <module>()
----> 1 function1()

<ipython-input-1-a9f4faeeeb0c> in function1()
      1 def function1():
----> 2     function2()
      3

<ipython-input-2-c65e19d6b45b> in function2()
      1 def function2():
----> 2     raise Exception('An exception occurred')

Exception: An exception   occurred
```

Traceback Details

The traceback shows the type of exception that occurred (`Exception`) followed by the complete function call stack that led to the raise point. The stack's bottom function call is listed *first* and the top is *last*, so the interpreter displays the following text as a reminder:

```
Traceback (most recent call last)
```

In this traceback, the following text indicates the bottom of the function-call stack—the `function1` call in snippet `[3]` (indicated by `ipython-input-3`):

```
<ipython-input-3-c0b3cafe2087> in <module>()
----> 1 function1()
```

Next, we see that `function1` called `function2` from line `2` in snippet `[1]`:

```
<ipython-input-1-a9f4faeeeb0c> in function1()
      1 def function1():
----> 2     function2()
      3
```

Finally, we see the *raise point*—in this case, line 2 in snippet `[2]` raised the exception:

```
<ipython-input-2-c65e19d6b45b> in function2()
      1 def function2():
----> 2     raise Exception('An exception occurred')
```

Stack Unwinding

In our previous exception-handling examples, the raise point occurred in a `try` suite, and the exception was handled in one of the `try` statement's corresponding `except` handlers. When an exception is *not* caught in a given function, **stack unwinding** occurs. Let's consider stack unwinding in the context of this example:

- In `function2`, the `raise` statement raises an exception. This is not in a `try` suite, so `function2` terminates, its stack frame is removed from the function-call stack, and control returns to the statement in `function1` that called `function2`.
- In `function1`, the statement that called `function2` is not in a `try` suite, so `function1` terminates, its stack frame is removed from the function-call stack, and control returns to the statement that called `function1`—snippet `[3]` in the IPython session.
- The call in snippet `[3]` call is not in a `try` suite, so that function call terminates. Because the exception was not caught (known as an **uncaught exception**), IPython displays the traceback, then awaits your next input. If this occurred in a typical script, the script would terminate.[o]

[o] In more advanced applications that use threads, an uncaught exception terminates only the thread in which the exception occurs, not necessarily the entire application.

Tip for Reading Tracebacks

You'll often call functions and methods that belong to libraries of code you did not

write. Sometimes those functions and methods raise exceptions. When reading a traceback, start from the end of the traceback and read the error message first. Then, read upward through the traceback, looking for the first line that indicates code you wrote in your program. Typically, this is the location in your code that led to the exception.

Exceptions in `finally` Suites

Raising an exception in a `finally` suite can lead to subtle, hard-to-find problems. If an exception occurs and is not processed by the time the `finally` suite executes, stack unwinding occurs. If the `finally` suite raises a *new* exception that the suite does not catch, the first exception is *lost*, and the *new* exception is passed to the next enclosing `try` statement. For this reason, a `finally` suite should always enclose in a `try` statement any code that may raise an exception, so that the exceptions will be processed within that suite.

9.12 INTRO TO DATA SCIENCE: WORKING WITH CSV FILES

Throughout this book, you'll work with many datasets as we present data-science concepts. **CSV (comma-separated values)** is a particularly popular file format. In this section, we'll demonstrate CSV file processing with a Python Standard Library module and pandas.

9.12.1 Python Standard Library Module `csv`

The **`csv` module**[1] provides functions for working with CSV files. Many other Python libraries also have built-in CSV support.

[1] `ttps://docs.python.org/3/library/csv.html`.

Writing to a CSV File

Let's create an `accounts.csv` file using CSV format. The `csv` module's documentation recommends opening CSV files with the additional keyword argument `newline=''` to ensure that newlines are processed properly:

lick here to view code image

```
In [1]: import csv

In [2]: with open('accounts.csv', mode='w',   newline='') as   accounts:
```

```
...:     writer = csv.writer(accounts)
...:     writer.writerow([100,   'Jones', 24.98])
...:     writer.writerow([200,   'Doe', 345.67])
...:     writer.writerow([300,   'White', 0.00])
...:     writer.writerow([400,   'Stone', -42.16])
...:     writer.writerow([500,   'Rich', 224.62])
...:
```

The **`.csv` file extension** indicates a CSV-format file. The `csv` module's **`writer` function** returns an object that writes CSV data to the specified file object. Each call to the `writer`'s **`writerow` method** receives an iterable to store in the file. Here we're using lists. By default, `writerow` delimits values with commas, but you can specify custom delimiters.[2] After the preceding snippet, `accounts.csv` contains:

[2] ttps://docs.python.org/3/library/csv.html#csv-fmt-params.

```
100,Jones,24.98
200,Doe,345.67
300,White,0.00
400,Stone,-42.16
500,Rich,224.62
```

CSV files generally do not contain spaces after commas, but some people use them to enhance readability. The `writerow` calls above can be replaced with one **`writerows`** call that outputs a comma-separated list of iterables representing the records.

If you write data that contains commas within a given string, `writerow` encloses that string in double quotes. For example, consider the following Python list:

```
[100, 'Jones, Sue', 24.98]
```

The single-quoted string `'Jones, Sue'` contains a comma separating the last name and first name. In this case, `writerow` would output the record as

```
100,"Jones, Sue",24.98
```

The quotes around `"Jones, Sue"` indicate that this is a *single* value. Programs reading this from a CSV file would break the record into *three* pieces—`100`, `'Jones, Sue'` and `24.98`.

h

Reading from a CSV File

Now let's read the CSV data from the file. The following snippet reads records from the file `accounts.csv` and displays the contents of each record, producing the same output we showed earlier:

lick here to view code image

```
In [3]: with open('accounts.csv', 'r',   newline='') as accounts:
   ...:     print(f'{"Account":<10}{"Name":<10}{"Balance":>10}')
   ...:     reader = csv.reader(accounts)
   ...:     for record in reader:
   ...:         account, name, balance = record
   ...:         print(f'{account:<10}{name:<10}{balance:>10}')
   ...:
Account   Name         Balance
100       Jones          24.98
200       Doe           345.67
300       White            0.0
400       Stone         -42.16
500       Rich          224.62
```

The `csv` module's **`reader` function** returns an object that reads CSV-format data from the specified file object. Just as you can iterate through a file object, you can iterate through the `reader` object one record of comma-delimited values at a time. The preceding `for` statement returns each `record` as a list of values, which we unpack into the variables `account`, `name` and `balance`, then display.

Caution: Commas in CSV Data Fields

Be careful when working with strings containing embedded commas, such as the name `'Jones, Sue'`. If you accidentally enter this as the two strings `'Jones'` and `'Sue'`, then `writerow` would, of course, create a CSV record with *four* fields, not *three*. Programs that read CSV files typically expect every record to have the *same* number of fields; otherwise, problems occur. For example, consider the following two lists:

```
[100, 'Jones', 'Sue', 24.98]
[200, 'Doe'  , 345.67]
```

The first list contains *four* values and the second contains only *three*. If these two records were written into the CSV file, then read into a program using the previous snippet, the following statement would fail when we attempt to unpack the four-field `record` into only three variables:

C

```
    account, name, balance = record
```

Caution: Missing Commas and Extra Commas in CSV Files

Be careful when preparing and processing CSV files. For example, suppose your file is composed of records, each with *four* comma-separated `int` values, such as:

```
    100,85,77,9
```

If you accidentally omit one of these commas, as in:

```
    100,8577,9
```

then the record has only *three* fields, one with the invalid value `8577`.

If you put two adjacent commas where only one is expected, as in:

```
    100,85,,77,9
```

then you have *five* fields rather than *four*, and one of the fields erroneously would be *empty*. Each of these comma-related errors could confuse programs trying to process the record.

9.12.2 Reading CSV Files into Pandas `DataFrame`s

In the Intro to Data Science sections in the previous two chapters, we introduced many pandas fundamentals. Here, we demonstrate pandas' ability to load files in CSV format, then perform some basic data-analysis tasks.

Datasets

In the data-science case studies, we'll use various free and open datasets to demonstrate machine learning and natural language processing concepts. There's an enormous variety of free datasets available online. The popular **Rdatasets repository** provides links to over 1100 free datasets in comma-separated values (CSV) format. These were originally provided with the R programming language for people learning about and developing statistical software, though they are not specific to R. They are now available on GitHub at:

```
ttps://vincentarelbundock.github.io/Rdatasets/datasets.html
```

This repository is so popular that there's a **`pydataset` module** specifically for accessing Rdatasets. For instructions on installing `pydataset` and accessing datasets with it, see:

```
ttps://github.com/iamaziz/PyDataset
```

Another large source of datasets is:

```
ttps://github.com/awesomedata/awesome-public-datasets
```

A commonly used machine-learning dataset for beginners is the **Titanic disaster dataset**, which lists all the passengers and whether they survived when the ship *Titanic* struck an iceberg and sank April 14–15, 1912. We'll use it here to show how to load a dataset, view some of its data and display some descriptive statistics. We'll dig deeper into a variety of popular datasets in the data-science chapters later in the book.

Working with Locally Stored CSV Files

You can load a CSV dataset into a `DataFrame` with the pandas function **`read_csv`**. The following loads and displays the CSV file `accounts.csv` that you created earlier in this chapter:

lick here to view code image

```
In [1]: import pandas as pd

In [2]: df = pd.read_csv('accounts.csv',
   ...:                  names=['account', 'name', 'balance'])
   ...:

In [3]: df
Out[3]:
   account   name  balance
0      100  Jones    24.98
1      200    Doe   345.67
2      300  White     0.00
3      400  Stone   -42.16
4      500   Rich   224.62
```

The `names` argument specifies the `DataFrame`'s column names. Without this

argument, `read_csv` assumes that the CSV file's first row is a comma-delimited list of column names.

To save a `DataFrame` to a file using CSV format, call `DataFrame` method **`to_csv`**:

lick here to view code image

```
In [4]: df.to_csv('accounts_from_dataframe.csv',    index=False)
```

The `index=False` keyword argument indicates that the row names (`0`–`4` at the left of the `DataFrame`'s output in snippet `[3]`) are not written to the file. The resulting file contains the column names as the first row:

```
account,name,balance
100,Jones,24.98
200,Doe,345.67
300,White,0.0
400,Stone,-42.16
500,Rich,224.62
```

9.12.3 Reading the Titanic Disaster Dataset

The Titanic disaster dataset is one of the most popular machine-learning datasets. The dataset is available in many formats, including CSV.

Loading the Titanic Dataset via a URL

If you have a URL representing a CSV dataset, you can load it into a `DataFrame` with `read_csv`. Let's load the Titanic Disaster dataset directly from GitHub:

lick here to view code image

```
In [1]: import pandas as pd

In [2]: titanic = pd.read_csv('https://vincentarelbundock.github.io/'    +
   ...:     'Rdatasets/csv/carData/TitanicSurvival.csv')
   ...:
```

Viewing Some of the Rows in the Titanic Dataset

This dataset contains over 1300 rows, each representing one passenger. According to Wikipedia, there were approximately 1317 passengers and 815 of them died.[3] For large

C

datasets, displaying the `DataFrame` shows only the first 30 rows, followed by "..." and the last 30 rows. To save space, let's view the first five and last five rows with `DataFrame` methods **`head`** and **`tail`**. Both methods return five rows by default, but you can specify the number of rows to display as an argument:

[3] ttps://en.wikipedia.org/wiki/Passengers_of_the_RMS_Titanic.

lick here to view code image

```
In [3]: pd.set_option('precision',   2)  # format for   floating-point val

 n [4]: titanic.head()
Out[4]:
                        Unnamed: 0 survived     sex    age   passengerClas
     Allen, Miss. Elisabeth Walton      yes  female    29.00            1s
    Allison, Master. Hudson Trevor      yes    male     0.92            1s
      Allison, Miss. Helen Loraine       no  female     2.00            1s
   Allison, Mr. Hudson Joshua Crei       no    male    30.00            1s
   Allison, Mrs. Hudson J C (Bessi       no  female    25.00            1s

 n [5]: titanic.tail()
Out[5]:
                     Unnamed: 0 survived     sex    age    passengerClass
1304       Zabour, Miss. Hileni       no  female    14.50              3rd
1305     Zabour, Miss. Thamine        no  female      NaN              3rd
1306  Zakarian, Mr. Mapriededer       no    male    26.50              3rd
1307        Zakarian, Mr. Ortin       no    male    27.00              3rd
1308         Zimmerman, Mr. Leo       no    male    29.00              3rd
```

Note that pandas adjusts each column's width, based on the widest value in the column or based on the column name, whichever is wider. Also, note the value in the age column of row `1305` is `NaN` (not a number), indicating a missing value in the dataset.

Customizing the Column Names

The first column in this dataset has a strange name (`'Unnamed: 0'`). We can clean that up by setting the column names. Let's change `'Unnamed: 0'` to `'name'` and let's shorten `'passengerClass'` to `'class'`:

lick here to view code image

```
In [6]: titanic.columns = ['name',   'survived', 'sex', 'age', 'class']

In [7]: titanic.head()
Out[7]:
```

```
                              name survived     sex    age   class
0    Allen, Miss. Elisabeth Walton      yes  female    29.00   1st
1   Allison, Master. Hudson Trevor      yes    male     0.92   1st
2     Allison, Miss. Helen Loraine       no  female     2.00   1st
3  Allison, Mr. Hudson Joshua Crei       no    male    30.00   1st
4  Allison, Mrs. Hudson J C (Bessi       no  female    25.00   1st
```

9.12.4 Simple Data Analysis with the Titanic Disaster Dataset

Now, you can use pandas to perform some simple analysis. For example, let's look at some descriptive statistics. When you call `describe` on a `DataFrame` containing both numeric and non-numeric columns, `describe` calculates these statistics *only for the numeric columns*—in this case, just the `age` column:

```
In [8]: titanic.describe()
Out[8]:
             age
count  1046.00
mean     29.88
std      14.41
min       0.17
25%      21.00
50%      28.00
75%      39.00
max      80.00
```

Note the discrepancy in the `count` (`1046`) vs. the dataset's number of rows (1309—the last row's index was `1308` when we called `tail`). Only `1046` (the `count` above) of the records contained an age value. The rest were *missing* and marked as `NaN`, as in row `1305`. When performing calculations, Pandas *ignores missing data (*`NaN`*) by default*. For the `1046` people with valid ages, the average (`mean`) age was `29.88` years old. The youngest passenger (`min`) was just over two months old (`0.17 * 12` is `2.04`), and the oldest (`max`) was 80. The median age was `28` (indicated by the `50%` quartile). The `25%` quartile is the median age in the first half of the passengers (sorted by age), and the `75%` quartile is the median of the second half of passengers.

Let's say you want to determine some statistics about people who survived. We can compare the `survived` column to `'yes'` to get a new `Series` containing `True`/`False` values, then use `describe` to summarize the results:

lick here to view code image

```
In [9]: (titanic.survived == 'yes').describe()
```

C

```
Out[9]:
count      1309
unique        2
top       False
freq        809
Name: survived, dtype: object
```

For non-numeric data, `describe` displays different descriptive statistics:

- `count` is the total number of items in the result.
- `unique` is the number of unique values (`2`) in the result—`True` (survived) and `False` (died).
- `top` is the most frequently occurring value in the result.
- `freq` is the number of occurrences of the `top` value.

9.12.5 Passenger Age Histogram

Visualization is a nice way to get to know your data. Pandas has many built-in visualization capabilities that are implemented with Matplotlib. To use them, first enable Matplotlib support in IPython:

```
In [10]: %matplotlib
```

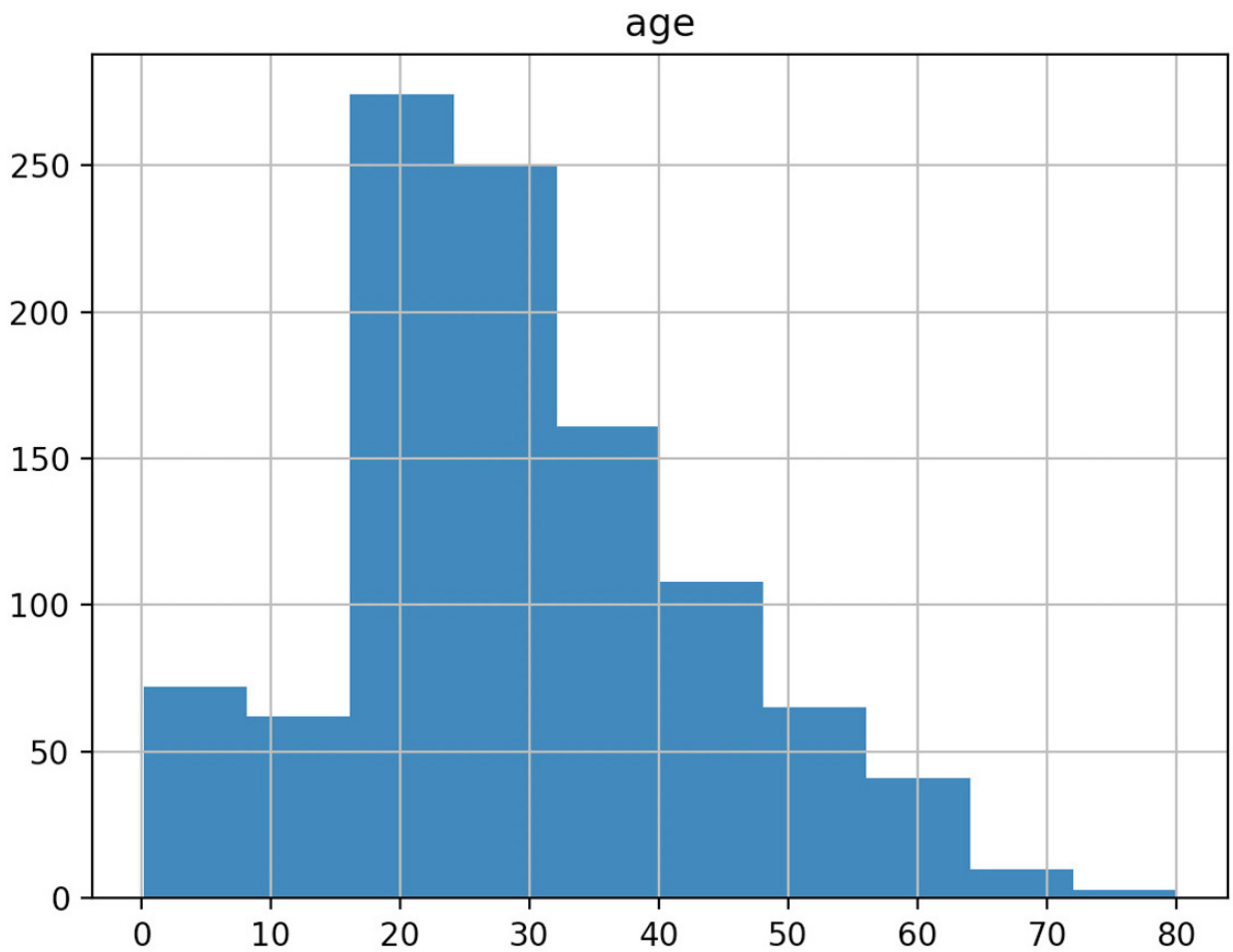
A histogram visualizes the distribution of numerical data over a range of values. A `DataFrame`'s **`hist`** method automatically analyzes each numerical column's data and produces a corresponding histogram. To view histograms of each numerical data column, call `hist` on your `DataFrame`:

lick here to view code image

```
In [11]: histogram = titanic.hist()
```

The Titanic dataset contains only one numerical data column, so the diagram shows one histogram for the age distribution. For datasets with multiple numerical columns, `hist` creates a separate histogram for each numerical column.

C



9.13 WRAP-UP

In this chapter, we introduced text-file processing and exception handling. Files are used to store data persistently. We discussed file objects and mentioned that Python views a file as a sequence of characters or bytes. We also mentioned the standard file objects that are automatically created for you when a Python program begins executing.

We showed how to create, read, write and update text files. We considered several popular file formats—plain text, JSON (JavaScript Object Notation) and CSV (comma-separated values). We used the built-in `open` function and the `with` statement to open a file, write to or read from the file and automatically close the file to prevent resource leaks when the `with` statement terminates. We used the Python Standard Library's `json` module to serialize objects into JSON format and store them in a file, load JSON objects from a file, deserialize them into Python objects and pretty-print a JSON object for readability.

We discussed how exceptions indicate execution-time problems and listed the various exceptions you've already seen. We showed how to deal with exceptions by wrapping code in `try` statements that provide `except` clauses to handle specific types of exceptions that may occur in the `try` suite, making your programs more robust and fault-tolerant.

e discussed the `try` statement's `finally` clause for executing code if program flow entered the corresponding `try` suite. You can use either the `with` statement or a `try` statement's `finally` clause for this purpose—we prefer the `with` statement.

In the Intro to Data Science section, we used both the Python Standard Library's `csv` module and capabilities of the pandas library to load, manipulate and store CSV data. Finally, we loaded the Titanic disaster dataset into a pandas `DataFrame`, changed some column names for readability, displayed the `head` and `tail` of the dataset, and performed simple analysis of the data. In the next chapter, we'll discuss Python's object-oriented programming capabilities.

W

10. Object-Oriented Programming

Objectives

In this chapter, you'll:

- Create custom classes and objects of those classes.
- Understand the benefits of crafting valuable classes.
- Control access to attributes.
- Appreciate the value of object orientation.
- Use Python special methods `__repr__`, `__str__` and `__format__` to get an object's string representations.
- Use Python special methods to overload (redefine) operators to use them with objects of new classes.
- Inherit methods, properties and attributes from existing classes into new classes, then customize those classes.
- Understand the inheritance notions of base classes (superclasses) and derived classes (subclasses).
- Understand duck typing and polymorphism that enable "programming in the general."
- Understand class `object` from which all classes inherit fundamental capabilities.
- Compare composition and inheritance.
- Build test cases into docstrings and run these tests with `doctest`,
- Understand namespaces and how they affect scope.

Outline

10.1 INTRODUCTION

ection 1.2 introduced the basic terminology and concepts of object-oriented programming. Everything in Python is an object, so you've been using objects constantly throughout this book. Just as houses are built from blueprints, objects are built from classes—one of the core technologies of object-oriented programming. Building a new object from even a large class is simple—you typically write one statement.

Crafting Valuable Classes

You've already used lots of classes created by other people. In this chapter you'll create your own *custom* classes. You'll focus on "crafting valuable classes" that help you meet the requirements of the applications you'll build. You'll use object-oriented programming with its core technologies of classes, objects, inheritance and polymorphism. Software applications are becoming larger and more richly functional. Object-oriented programming makes it easier for you to design, implement, test, debug and update such edge-of-the-practice applications. Read ections 10.1 through 0.9 for a code-intensive introduction to these technologies. Most people can skip ections 10.10 through 0.15, which provide additional perspectives on these technologies and present additional related features.

Class Libraries and Object-Based Programming

The vast majority of object-oriented programming you'll do in Python is **object-based programming** in which you primarily create and use objects of *existing* classes. You've been doing this throughout the book with built-in types like `int`, `float`, `str`, `list`, `tuple`, `dict`

nd `set`, with Python Standard Library types like `Decimal`, and with NumPy `array`s, Matplotlib `Figure`s and `Axes`, and pandas `Series` and `DataFrame`s.

To take maximum advantage of Python you must familiarize yourself with lots of preexisting classes. Over the years, the Python open-source community has crafted an enormous number of valuable classes and packaged them into class libraries. This makes it easy for you to reuse existing classes rather than "reinventing the wheel." Widely used open-source library classes are more likely to be thoroughly tested, bug free, performance tuned and portable across a wide range of devices, operating systems and Python versions. You'll find abundant Python libraries on the Internet at sites like GitHub, BitBucket, Source Forge and more—most easily installed with `conda` or `pip`. This is a key reason for Python's popularity. The vast majority of the classes you'll need are likely to be freely available in open-source libraries.

Creating Your Own Custom Classes

Classes are new data types. Each Python Standard Library class and third-party library class is a custom type built by someone else. In this chapter, you'll develop application-specific classes, like `CommissionEmployee`, `Time`, `Card`, `DeckOfCards` and more.

Most applications you'll build for your own use will commonly use either no custom classes or just a few. If you become part of a development team in industry, you may work on applications that contain hundreds, or even thousands, of classes. You can contribute your custom classes to the Python open-source community, but you are not obligated to do so. Organizations often have policies and procedures related to open-sourcing code.

Inheritance

Perhaps most exciting is the notion that new classes can be formed through inheritance and composition from classes in abundant class libraries. Eventually, software will be constructed predominantly from **standardized, reusable components** just as hardware is constructed from interchangeable parts today. This will help meet the challenges of developing ever more powerful software.

When creating a new class, instead of writing all new code, you can designate that the new class is to be formed initially by **inheriting** the attributes (variables) and methods (the class version of functions) of a previously defined **base class** (also called a **superclass**). The new class is called a **derived class** (or **subclass**). After inheriting, you then customize the derived class to meet the specific needs of your application. To minimize the customization effort, you should always try to inherit from the base class that's closest to your needs. To do that effectively, you should familiarize yourself with the class libraries that are geared to the kinds of applications you'll be building.

Polymorphism

We explain and demonstrate **polymorphism**, which enables you to conveniently program "in the general" rather than "in the specific." You simply send the *same* method call to objects possibly of many *different* types. Each object responds by "doing the right thing." So the same

a

ethod call takes on “many forms,” hence the term “poly-morphism.” We’ll explain how to implement polymorphism through inheritance and a Python feature called duck typing. We’ll explain both and show examples of each.

An Entertaining Case Study: Card-Shuffling-and-Dealing Simulation

You’ve already used a random-numbers-based die-rolling simulation and used those techniques to implement the popular dice game craps. Here, we present a card-shuffling-and-dealing simulation, which you can use to implement your favorite card games. You’ll use Matplotlib with attractive public-domain card images to display the full deck of cards both before and after the deck is shuffled.

Data Classes

Python 3.7’s new *data classes* help you build classes faster by using a more concise notation and by autogenerating portions of the classes. The Python community’s early reaction to data classes has been positive. As with any major new feature, it may take time before it’s widely used. We present class development with both the older and newer technologies.

Other Concepts Introduced in This Chapter

Other concepts we present include:

- How to specify that certain identifiers should be used only inside a class and not be accessible to clients of the class.
- Special methods for creating string representations of your classes’ objects and specifying how objects of your classes work with Python’s built-in operators (a process called *operator overloading*).
- An introduction to the Python exception class hierarchy and creating custom exception classes.
- Testing code with the Python Standard Library’s `doctest` module.
- How Python uses namespaces to determine the scopes of identifiers.

10.2 CUSTOM CLASS `ACCOUNT`

Let’s begin with a bank `Account` class that holds an account holder’s name and balance. An actual bank account class would likely include lots of other information, such as address, birth date, telephone number, account number and more. The `Account` class accepts deposits that increase the balance and withdrawals that decrease the balance.

10.2.1 Test-Driving Class `Account`

Each new class you create becomes a new *data type* that can be used to create objects. This is one reason why Python is said to be an **extensible language**. Before we look at class

m

`Account`'s definition, let's demonstrate its capabilities.

Importing Classes `Account` and `Decimal`

To use the new `Account` class, launch your IPython session from the `ch10` examples folder, then import class `Account`:

lick here to view code image

```
In [1]: from account import Account
```

Class `Account` maintains and manipulates the account balance as a `Decimal`, so we also import class `Decimal`:

lick here to view code image

```
In [2]: from decimal import Decimal
```

Create an `Account` Object with a Constructor Expression

To create a `Decimal` object, we can write:

```
value = Decimal('12.34')
```

This is known as a **constructor expression** because it builds and initializes an object of the class, similar to the way a house is constructed from a blueprint then painted with the buyer's preferred colors. Constructor expressions create new objects and initialize their data using argument(s) specified in parentheses. The parentheses following the class name are required, even if there are no arguments.

Let's use a constructor expression to create an `Account` object and initialize it with an account holder's name (a string) and balance (a `Decimal`):

lick here to view code image

```
In [3]: account1 = Account('John   Green', Decimal('50.00'))
```

Getting an `Account`'s Name and Balance

Let's access the `Account` object's `name` and `balance` attributes:

lick here to view code image

```
In [4]: account1.name
Out[4]: 'John Green'
```

C

```
In [5]: account1.balance
Out[5]: Decimal('50.00')
```

Depositing Money into an `Account`

An `Account`'s `deposit` method receives a positive dollar amount and adds it to the balance:

lick here to view code image

```
In [6]: account1.deposit(Decimal('25.53'))

In [7]: account1.balance
Out[7]: Decimal('75.53')
```

`Account` Methods Perform Validation

Class `Account`'s methods validate their arguments. For example, if a deposit amount is negative, `deposit` raises a `ValueError`:

lick here to view code image

```
In [8]: account1.deposit(Decimal('-123.45'))
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-8-27dc468365a7> in <module>()
----> 1 account1.deposit(Decimal('-123.45'))

~/Documents/examples/ch10/account.py in deposit(self, amount)
     21         # if amount is less than 0.00, raise an exception
     22         if amount < Decimal('0.00'):
---> 23             raise ValueError('Deposit amount must be positive.')
     24
     25         self.balance += amount

ValueError: Deposit amount must be positive.
```

10.2.2 `Account` Class Definition

Now, let's look at `Account`'s class definition, which is located in the file `account.py`.

Defining a Class

A class definition begins with the keyword **`class`** (line 5) followed by the class's name and a colon (`:`). This line is called the **class header**. The *Style Guide for Python Code* recommends that you begin each word in a multi-word class name with an uppercase letter (for example, `CommissionEmployee`). Every statement in a class's suite is indented.

lick here to view code image

```
1 # account.py
```

C

```
2 """Account class definition."""
3 from decimal import Decimal
4
5 class Account:
6     """Account class for maintaining a bank   account balance."""
7
```

Each class typically provides a descriptive docstring (line 6). When provided, it must appear in the line or lines immediately following the class header. To view any class's docstring in IPython, type the class name and a question mark, then press *Enter*:

lick here to view code image

```
In [9]: Account?
 nit signature: Account(name, balance)
Docstring:      Account class for maintaining a bank   account balance.
Init docstring: Initialize an Account object.
File:           ~/Documents/examples/ch10/account.py
Type:           type
```

The identifier `Account` is both the class name and the name used in a constructor expression to create an `Account` object and invoke the class's `__init__` method. For this reason, IPython's help mechanism shows both the class's docstring (`"Docstring:"`) and the `__init__` method's docstring (`"Init docstring:"`).

Initializing Account Objects: Method `__init__`

The constructor expression in snippet `[3]` from the preceding section:

lick here to view code image

```
account1 = Account('John Green', Decimal('50.00'))
```

creates a new object, then initializes its data by calling the class's **`__init__`** method. Each new class you create can provide an `__init__` method that specifies how to initialize an object's data attributes. Returning a value other than `None` from `__init__` results in a `TypeError`. Recall that `None` is returned by any function or method that does not contain a `return` statement. Class `Account`'s `__init__` method (lines 8–16) initializes an `Account` object's `name` and `balance` attributes if the `balance` is valid:

lick here to view code image

```
8     def __init__(self, name, balance):
9         """Initialize an Account   object."""
10
11         # if balance is less than 0.00, raise an exception
12         if balance < Decimal('0.00'):
13             raise ValueError('Initial balance   must be >= to 0.00.')
```

C

```
14
15          self.name    = name
16          self.balance    = balance
17
```

When you call a method for a specific object, Python implicitly passes a reference to that object as the method's first argument. For this reason, all methods of a class must specify at least one parameter. By convention most Python programmers call a method's first parameter `self`. A class's methods must use that reference (`self`) to access the object's attributes and other methods. Class `Account`'s `__init__` method also specifies parameters for the `name` and `balance`.

The `if` statement *validates* the `balance` parameter. If `balance` is less than `0.00`, `__init__` raises a `ValueError`, which terminates the `__init__` method. Otherwise, the method creates and initializes the new `Account` object's `name` and `balance` attributes.

When an object of class `Account` is created, it does not yet have any attributes. They're added *dynamically* via assignments of the form:

```
self.attribute_name = value
```

Python classes may define many **special methods**, like `__init__`, each identified by leading and trailing double-underscores (`__`) in the method name. Python class **`object`**, which we'll discuss later in this chapter, defines the special methods that are available for *all* Python objects.

Method `deposit`

The `Account` class's `deposit` method adds a positive `amount` to the account's `balance` attribute. If the `amount` argument is less than `0.00`, the method raises a `ValueError`, indicating that only positive deposit `amount`s are allowed. If the `amount` is valid, line 25 adds it to the object's `balance` attribute.

lick here to view code image

```
18      def deposit(self, amount):
19          """Deposit money to the    account."""
20
21          # if amount is less than 0.00, raise an exception
22          if amount < Decimal('0.00'):
23              raise ValueError('amount must be    positive.')
24
25          self.balance    += amount
```

10.2.3 Composition: Object References as Members of Classes

An `Account` *has a* `name`, and an `Account` *has a* `balance`. Recall that "everything in Python

C

is an object." This means that an object's attributes are references to objects of other classes. For example, an `Account` object's `name` attribute is a reference to a string object and an `Account` object's `balance` attribute is a reference to a `Decimal` object. Embedding references to objects of other types is a form of software reusability known as **composition** and is sometimes referred to as the **"has a" relationship**. Later in this chapter, we'll discuss *inheritance*, which establishes *"is a"* relationships.

10.3 CONTROLLING ACCESS TO ATTRIBUTES

Class `Account`'s methods validate their arguments to ensure that the `balance` is *always* valid—that is, always greater than or equal to `0.00`. In the previous example, we used the attributes `name` and `balance` only to *get* the values of those attributes. It turns out that we also can use those attributes to *modify* their values. Consider the `Account` object in the following IPython session:

lick here to view code image

```
In [1]: from account import Account

In [2]: from decimal import Decimal

In [3]: account1 = Account('John   Green', Decimal('50.00'))

In [4]: account1.balance
Out[4]: Decimal('50.00')
```

Initially, `account1` contains a valid balance. Now, let's set the `balance` attribute to an *invalid* negative value, then display the `balance`:

lick here to view code image

```
In [5]: account1.balance = Decimal('-1000.00')

In [6]: account1.balance
Out[6]: Decimal('-1000.00')
```

Snippet `[6]`'s output shows that `account1`'s `balance` is now negative. As you can see, unlike methods, data attributes cannot validate the values you assign to them.

Encapsulation

A class's **client code** is any code that uses objects of the class. Most object-oriented programming languages enable you to **encapsulate** (or *hide*) an object's data from the client code. Such data in these languages is said to be *private data*.

Leading Underscore (_) Naming Convention

Python does *not* have private data. Instead, you use *naming conventions* to design classes

C

that encourage correct use. By convention, Python programmers know that any attribute name beginning with an underscore (_) is for a class's *internal use only*. Client code should use the class's methods and—as you'll see in the next section—the class's properties to interact with each object's internal-use data attributes. Attributes whose identifiers do *not* begin with an underscore (_) are considered *publicly accessible* for use in client code. In the next section, we'll define a `Time` class and use these naming conventions. However, even when we use these conventions, attributes are always accessible.

10.4 PROPERTIES FOR DATA ACCESS

Let's develop a `Time` class that stores the time in 24-hour clock format with hours in the range 0–23, and minutes and seconds each in the range 0–59. For this class, we'll provide *properties*, which look like data attributes to client-code programmers, but control the manner in which they get and modify an object's data. This assumes that other programmers follow Python conventions to correctly use objects of your class.

10.4.1 Test-Driving Class `Time`

Before we look at class `Time`'s definition, let's demonstrate its capabilities. First, ensure that you're in the `ch10` folder, then `import` class `Time` from `timewithproperties.py`:

lick here to view code image

```
In [1]: from timewithproperties import Time
```

Creating a `Time` Object

Next, let's create a `Time` object. Class `Time`'s `__init__` method has `hour`, `minute` and `second` parameters, each with a default argument value of `0`. Here, we specify the `hour` and `minute`—`second` defaults to `0`:

lick here to view code image

```
In [2]: wake_up = Time(hour=6,    minute=30)
```

Displaying a `Time` Object

Class `Time` defines two methods that produce string representations of `Time` object. When you evaluate a variable in IPython as in snippet `[3]`, IPython calls the object's `__repr__` special method to produce a string representation of the object. Our `__repr__` implementation creates a string in the following format:

lick here to view code image

```
In [3]: wake_up
```

C

```
Out[3]: Time(hour=6, minute=30, second=0)
```

We'll also provide the `__str__` special method, which is called when an object is converted to a string, such as when you output the object with `print`. [1] Our `__str__` implementation creates a string in 12-hour clock format:

[1] If a class does not provide `__str__` and an object of the class is converted to a string, the classs `__repr__` method is called instead.

```
In [4]: print(wake_up)
6:30:00 AM
```

Getting an Attribute Via a Property

Class time provides `hour`, `minute` and `second` **properties**, which provide the convenience of data attributes for getting and modifying an object's data. However, as you'll see, properties are implemented as methods, so they may contain additional logic, such as specifying the format in which to return a data attribute's value or validating a new value before using it to modify a data attribute. Here, we get the `wake_up` object's `hour` value:

```
In [5]: wake_up.hour
Out[5]: 6
```

Though this snippet appears to simply get an `hour` data attribute's value, it's actually a call to an `hour` *method* that returns the value of a data attribute (which we named `_hour`, as you'll see in the next section).

Setting the `Time`

You can set a new time with the `Time` object's `set_time` method. Like method `__init__`, method `set_time` provides `hour`, `minute` and `second` parameters, each with a default of `0`:

lick here to view code image

```
In [6]: wake_up.set_time(hour=7,   minute=45)

In [7]: wake_up
Out[7]: Time(hour=7, minute=45, second=0)
```

Setting an Attribute via a Property

Class `Time` also supports setting the `hour`, `minute` and `second` values individually via its properties. Let's change the `hour` value to `6`:

lick here to view code image

C

```
In [8]: wake_up.hour = 6

In [9]: wake_up
Out[9]: Time(hour=6, minute=45, second=0)
```

Though snippet [8] appears to simply assign a value to a data attribute, it's actually a call to an `hour` method that takes `6` as an argument. The method validates the value, then assigns it to a corresponding data attribute (which we named `_hour`, as you'll see in the next section).

Attempting to Set an Invalid Value

To prove that class `Time`'s properties *validate* the values you assign to them, let's try to assign an invalid value to the `hour` property, which results in a `ValueError`:

lick here to view code image

```
In [10]: wake_up.hour = 100
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-10-1fce0716ef14> in <module>()
----> 1 wake_up.hour = 100

~/Documents/examples/ch10/timewithproperties.py in   hour(self, hour)
     20         """Set   the hour."""
     21         if not (0 <= hour < 24):
---> 22             raise   ValueError(f'Hour ({hour})   must be 0-23')
     23
     24         self._hour = hour

ValueError: Hour (100) must   be 0-23
```

10.4.2 Class `Time` Definition

Now that we've seen class `Time` in action, let's look at its definition.

Class Time: `__init__` Method with Default Parameter Values

Class `Time`'s `__init__` method specifies `hour`, `minute` and `second` parameters, each with a default argument of `0`. Similar to class `Account`'s `__init__` method, recall that the `self` parameter is a reference to the `Time` object being initialized. The statements containing `self.hour`, `self.minute` and `self.second` *appear* to create `hour`, `minute` and `second` attributes for the new `Time` object (`self`). However, these statements actually call methods that implement the class's `hour`, `minute` and `second` *properties* (lines 13–50). Those methods then create attributes named `_hour`, `_minute` and `_second` that are meant for use only inside the class:

lick here to view code image

```
1 # timewithproperties.py
2 """Class Time with read-write   properties."""
```

C

```
3
4 class Time:
5     """Class Time with read-write   properties."""
6
7     def __init__(self, hour=0,   minute=0, second=0):
8         """Initialize each   attribute."""
9         self.hour   = hour  # 0-23
10         self.minute   = minute  # 0-59
11         self.second   = second  # 0-59
12
```

Class Time: `hour` Read-Write Property

Lines 13–24 define a *publicly accessible* **read-write property** named `hour` that manipulates a data attribute named `_hour`. The single-leading-underscore (`_`) naming convention indicates that client code should not access `_hour` directly. As you saw in the previous section's snippets `[5]` and `[8]`, properties look like data attributes to programmers working with `Time` objects. However, notice that properties are implemented as *methods*. Each property defines a *getter* method which *gets* (that is, returns) a data attribute's value and can *optionally* define a *setter* method which *sets* a data attribute's value:

lick here to view code image

```
13     @property
14     def hour(self):
15         """Return the hour."""
16         return self._hour
17
18     @hour.setter
19     def hour(self, hour):
20         """Set the hour."""
21         if not (0 <= hour < 24):
22             raise ValueError(f'Hour ({hour}) must be 0-23')
23
24         self._hour   = hour
25
```

The **`@property` decorator** precedes the property's *getter* method, which receives only a `self` parameter. Behind the scenes, a decorator adds code to the decorated function—in this case to make the `hour` function work with attribute syntax. The *getter* method's name is the property name. This *getter* method returns the `_hour` data attribute's value. The following client-code expression invokes the *getter* method:

```
wake_up.hour
```

You also can use the *getter* method inside the class, as you'll see shortly.

A decorator of the form **`@property_name.setter`** (in this case, `@hour.setter`) precedes the property's *setter* method. The method receives two parameters—`self` and a parameter (`hour`) representing the value being assigned to the property. If the `hour`

C

parameter's value is *valid*, this method assigns it to the `self` object's `_hour` attribute; otherwise, the method raises a `ValueError`. The following client-code expression invokes the *setter* by assigning a value to the property:

```
wake_up.hour = 8
```

We also invoked this `setter` inside the class at line 9 of `__init__`:

```
self.hour = hour
```

Using the *setter* enabled us to *validate* `__init__`'s `hour` argument *before* creating and initializing the object's `_hour` attribute, which occurs the *first* time the `hour` property's *setter* executes as a result of line 9. A **read-write property** has both a *getter* and a *setter*. A **read-only property** has only a *getter*.

Class Time: `minute` and `second` Read-Write Properties

Lines 26–37 and 39–50 define read-write `minute` and `second` properties. Each property's `setter` ensures that its second argument is in the range `0–59` (the valid range of values for minutes and seconds):

lick here to view code image

```
26      @property
27      def minute(self):
28          """Return the   minute."""
29          return self._minute
30
31      @minute.setter
32      def minute(self, minute):
33          """Set the minute."""
34          if not (0 <= minute <   60):
35              raise ValueError(f'Minute ({minute}) must be 0-59')
36
37          self._minute   = minute
38
39      @property
40      def second(self):
41          """Return the   second."""
42          return self._second
43
44      @second.setter
45      def second(self, second):
46          """Set the second."""
47          if not (0 <= second <   60):
48              raise ValueError(f'Second ({second}) must be 0-59')
49
50          self._second   = second
51
```

Class Time: Method `set_time`

C

We provide method `set_time` as a convenient way to change *all three* attributes with a *single* method call. Lines 54–56 invoke the *setters* for the `hour`, `minute` and `second` properties:

lick here to view code image

```
52     def set_time(self, hour=0,   minute=0, second=0):
53         """Set values of hour, minute, and   second."""
54         self.hour   = hour
55         self.minute   = minute
56         self.second   = second
57
```

Class Time: Special Method `__repr__`

When you pass an object to built-in function `repr`—which happens implicitly when you evaluate a variable in an IPython session—the corresponding class's **`__repr__` special method** is called to get a string representation of the object:

lick here to view code image

```
58     def __repr__(self):
59         """Return Time string for   repr()."""
60         return (f'Time(hour={self.hour}, minute={self.minute}, '   +
61                 f'second={self.second})')
62
```

The Python documentation indicates that `__repr__` returns the "official" string representation of the object. Typically this string looks like a constructor expression that creates and initializes the object, [2] as in:

[2] `ttps://docs.python.org/3/reference/datamodel.html`.

```
'Time(hour=6, minute=30, second=0)'
```

which is similar to the constructor expression in the previous section's snippet `[2]`. Python has a built-in function **`eval`** that could receive the preceding string as an argument and use it to create and initialize a `Time` object containing values specified in the string.

Class Time: Special Method `__str__`

For our class `Time` we also define the **`__str__`** special method. This method is called implicitly when you convert an object to a string with the built-in function `str`, such as when you `print` an object or call `str` explicitly. Our implementation of `__str__` creates a string in 12-hour clock format, such as `'7:59:59 AM'` or `'12:30:45 PM'`:

lick here to view code image

```
    def __str__(self):
        """Print Time in 12-hour clock   format."""
        return (('12' if self.hour in (0, 12) else str(self.hour % 12)) +
                f':{self.minute:0>2}:{self.second:0>2}' +
                (' AM' if self.hour < 12 else ' PM'))
```

10.4.3 Class `Time` Definition Design Notes

Let's consider some class-design issues in the context of our `Time` class.

Interface of a Class

Class `Time`'s properties and methods define the class's **public interface**—that is, the set of properties and methods programmers should use to interact with objects of the class.

Attributes Are Always Accessible

Though we provided a well-defined interface, Python does *not* prevent you from directly manipulating the data attributes `_hour`, `_minute` and `_second`, as in:

lick here to view code image

```
In [1]: from timewithproperties import Time

In [2]: wake_up = Time(hour=7,   minute=45, second=30)

In [3]: wake_up._hour
Out[3]: 7

In [4]: wake_up._hour = 100

In [5]: wake_up
Out[5]: Time(hour=100, minute=45, second=30)
```

After snippet `[4]`, the `wake_up` object contains *invalid* data. Unlike many other object-oriented programming languages, such as C++, Java and C#, data attributes in Python cannot be hidden from client code. The Python tutorial says, "**nothing in Python makes it possible to enforce data hiding—it is all based upon convention**." [3]

[3] `ttps://docs.python.org/3/tutorial/classes.html#random-remarks`.

Internal Data Representation

We chose to represent the time as three integer values for hours, minutes and seconds. It would be perfectly reasonable to represent the time internally as the number of seconds since midnight. Though we'd have to reimplement the properties `hour`, `minute` and `second`, programmers could use the *same* interface and get the *same* results without being aware of these changes. We leave it to you to make this change and show that client code using `Time` objects does not need to change.